INFRAESTRUCTURES PER AL BIG DATA
(17244106, 17254118, 17924106)
HW2: Big Data Analytics with PySpark

- The deadline for the delivery of this exercise is **May 10th, 2026**.
- A ZIP file with the source code and a PDF report must be delivered.
- The source code may be submitted as **Jupyter/Colab notebooks and/or .py files**, provided it is clearly organized and executable.
- The submission of the source code and documentation must be done using Moodle.

Task Description: New York City generates massive amounts of urban mobility data every day. In this project, you will design and implement a complete big data analytics pipeline using PySpark to process, clean, analyse, and model taxi trip data from the NYC TLC Trip Record Data dataset.

Unlike a basic Python exercise, this project focuses on big data engineering and analytics concepts, including:

- Distributed data loading and preprocessing with Spark
- Working with large public datasets in Parquet and/or CSV format
- Data cleaning and validation at scale
- Feature engineering using PySpark
- Exploratory data analysis with Spark SQL and DataFrames
- Performance optimization in Spark
- Aggregation and window-based analytics
- A final advanced task using Spark MLlib or anomaly analysis

The goal is not only to write PySpark code, but to understand how Spark processes large data, how distributed transformations differ from local processing, and how data engineering decisions affect scalability and performance.

Learning Objectives: By completing this project, you will be able to:

- Load and process a large public dataset using PySpark.
- Apply cleaning and feature engineering in distributed environments.
- Use Spark DataFrames and Spark SQL for large-scale analysis.
- Design meaningful analytical questions on mobility data.
- Apply performance techniques such as repartitioning, caching, and plan inspection.
- Use window functions for advanced analytics.
- Implement one advanced task with Spark MLlib or anomaly detection.
- Communicate findings clearly in a technical report.

Team Structure: Each group must consist of 3 students and assign the following roles:

- Data Engineer: Responsible for data ingestion, schema inspection, cleaning, and storage.

- Analytics Engineer: Responsible for Spark SQL, aggregations, business questions, and visual summaries.
- ML/Performance Engineer: Responsible for feature preparation, advanced task, performance tuning, and execution analysis.

The final report must clearly indicate the contribution of each member.

Working Environment: Groups may use Google Colab for this assignment. Because the free edition has limited memory and session resources, groups are advised to work with a moderate subset of the NYC TLC dataset, such as 2–3 months of records, and justify their selection in the report.

Dataset Description: You must use the NYC TLC Trip Record Data as the main dataset. The analysis may be enriched with the taxi zone lookup table when useful. The minimum recommended workload is at least 3 monthly files. Stronger groups may extend the analysis to 6 or more months if their environment supports it.

Common attributes available in the dataset include pickup datetime, drop-off datetime, pickup location ID, drop-off location ID, passenger count, trip distance, fare amount, payment type, and total amount.

Groups must download the dataset from the official NYC TLC Trip Record Data page. Use the Yellow Taxi Trip Records (PARQUET) files and the Taxi Zone Lookup Table. For the mandatory part of the assignment, each group should download 3 monthly Yellow Taxi files from the same year, plus the lookup table. Students who want to extend the analysis may use more months if their environment supports it. The official TLC page provides the monthly Parquet files and trip-data documentation.

Example: This is simple, consistent, and enough for a 3-week Spark project. The official TLC site provides monthly Yellow Taxi Parquet files, and the Taxi Zones dataset is the standard lookup for the pickup and dropoff location IDs used in TLC trip records.

- *yellow_tripdata_2024-01.parquet*
- *yellow_tripdata_2024-02.parquet*
- *yellow_tripdata_2024-03.parquet*
- *taxi_zone_lookup.csv*

<u>Steps to Complete the Assignment:</u>

1. Data Acquisition and Loading

- Download the selected NYC TLC trip data files.
- Load them into PySpark using SparkSession.
- Read the data using the correct format, preferably Parquet.
- Inspect the schema and data types.
- Merge multiple monthly files into one Spark DataFrame.

2. Data Cleaning and Validation

- Identify and handle problematic records such as null values in key fields.
- Remove or justify records with zero or negative trip distance, duration, or fare.

- Inspect unrealistic passenger counts and inconsistent pickup/drop-off times.
- Explain and justify all cleaning decisions in the report.

3. Feature Engineering

- Create trip duration in minutes.
- Create pickup hour, pickup weekday, and month.
- Compute average speed.
- Create fare efficiency indicators such as fare per kilometer.
- Add any other useful derived feature and justify it.

4. Exploratory Data Analysis with Spark Use Spark DataFrames and/or Spark SQL to answer the following questions:

- What are the busiest pickup hours of the day?
- What are the busiest pickup and drop-off zones?
- How does trip demand vary by weekday?
- Which zones generate the highest total revenue?
- What is the average trip distance by hour?
- What is the average fare amount by weekday?
- Which payment type is most frequent?
- Which zones have the longest average trip duration?
- Which zones have the shortest but most frequent trips?
- Identify trips with unrealistic average speed.
- Identify records with suspiciously high fare for short distance.
- Rank the top 10 pickup zones per month.
- Compare daytime versus nighttime trip behaviour.
- Find patterns of demand concentration over time.

5. Use of Window Functions You must implement at least two analyses using window functions, for example:

- Ranking the top pickup zones per day or per month.
- Ranking top revenue zones by period.
- Computing moving averages of trip counts.
- Comparing each zone against the city average.

6. Performance and Optimization For at least three operations or queries, do the following:

- Run the analysis normally.
- Inspect the execution plan using function ***explain()***.
- Apply one optimization strategy such as ***cache(), repartition(),*** column pruning, or filtering before joins.
- Run the analysis again.
- Explain what improved and why.

7. Advanced Task Each group must complete one of the following options:

- **Option A** – Prediction with Spark MLlib: predict trip duration, fare amount, or fare category.
- **Option B** – Clustering: group zones or trips according to mobility behaviour.
- **Option C** – Anomaly Detection: detect suspicious or abnormal trips using justified rules or models.

8. Short Reflection In the report, include a short reflection (8–12 lines) discussing:

- Which step was the most computationally expensive and why.
- Which Spark optimization was the most useful.
- What was difficult about working with large data in distributed mode.

- What insights the dataset revealed about urban mobility.

Submission Guidelines

Each group must submit a single ZIP file containing:

1. Source code

- A notebook or .py file for loading and cleaning.
- A notebook or .py file for analytical queries.
- A notebook or .py file for the advanced task.
- Clear comments and section titles.
- Recommended filename: HW2_NYC_Taxi_GroupXX.zip

2. PDF Report The report must include:

- Project overview, objective, selected months, and group members.
- Data understanding: schema, size, and key attributes.
- Data cleaning: invalid records found and rules applied.
- Feature engineering: created features and justification.
- Query results and figures with short interpretation.
- At least two analyses based on window functions.
- Performance analysis for three operations before and after optimization.
- Advanced task methodology, results, and interpretation.
- Team contribution.

Submission Platform: Submit the ZIP file through Moodle.

Grading Criteria

- Data loading and cleaning correctness: 20%
- Feature engineering quality: 15%
- Analytical queries and insight quality: 25%
- Window functions and advanced Spark usage: 15%
- Performance analysis and optimization: 15%
- Report clarity, completeness, and teamwork: 10%

General Notes:

- Do not use Pandas for full dataset processing.
- Use Pandas only for small final aggregated tables if needed for plotting.
- Most of the work must be done in PySpark.
- Use meaningful comments and clean code organization.
- Validate whether your results make sense before writing conclusions.
- Every group member must understand the whole pipeline, not only their assigned part.

Use of Generative AI Tools:

- The use of Generative AI tools (e.g., ChatGPT, Copilot, etc.) is allowed only as a support tool for learning.
- You must understand and be able to explain all code submitted.
- Blindly copying AI-generated code without understanding it is strictly prohibited.
- During evaluation or oral review, any group member may be asked to explain any part of the submission.
- If you use AI tools, you must briefly indicate in your report how they were used.

- Academic integrity policies apply. Submissions that demonstrate lack of understanding may receive a reduced grade.

Guidelines for Students

1. Work as a real team

- Divide responsibilities, but make sure everyone understands the full workflow.
- Keep track of the selected data, cleaning rules, created features, analytical questions, and advanced task.

2. Start early

- Week 1: choose the data period, load data, inspect schema, clean records, and create features.
- Week 2: answer analytical questions, build figures, implement window functions, and start performance analysis.
- Week 3: complete the advanced task, improve the code, write the report, and review the final ZIP.

3. Keep your project organized

- Use a clean folder structure such as data/, notebooks/, scripts/, figures/, and report/.

4. Justify every decision

- Explain why rows were removed, why features are useful, why a model was selected, and why an optimization helped.

5. Make figures readable

- All plots should have a title, labelled axes, and readable legends when needed.

6. Focus on Spark

- The main logic must use Spark DataFrames, Spark SQL, PySpark functions, and Spark MLlib when applicable.

7. Prepare for possible oral questions

- Be ready to explain why Spark is better than local Pandas here, how transformations and actions differ, and what your optimization or model means.